BERT-NER: A TRANSFORMER-BASED APPROACH FOR NAMED ENTITY RECOGNITION

Navjeet Kaur
Department of Computer Science and Engineering
Apex Institute of Technology, Chandigarh University
Mohali, Punjab, India
navjeet.e16069@cumail.in

Ashish Saha
Department of Computer Science and Engineering
Apex Institute of Technology, Chandigarh University
Mohali, Punjab, India
21BCS11069@cuchd.in

Makul Swami
Department of Computer Science and Engineering
Apex Institute of Technology, Chandigarh University
Mohali, Punjab, India
21BCS3798@cuchd.in

Muskan Singh
Department of Computer Science and Engineering
Apex Institute of Technology, Chandigarh University
Mohali, Punjab, India
21BCS3815@cuchd.in

Ravi Dalal
Department of Computer Science and Engineering
Apex Institute of Technology, Chandigarh University
Mohali, Punjab, India
21BCS3811@cuchd.in

***Abstract*—The NER ("Named Entity Recognition") technique picks up on named entities including people, places, companies etc., that are in the unstructured data in a new language processing technique. But, it arises a fundamental issue of the lack of localization that cannot be feasible. In spite of the achieved top performance of supervised NER methods in resource- rich domains, they usually too slow and not so accurate in domains with little labeled data. Here, the work present a new model known as BERT-NER which is an exceptionally deep learning approach built on the BERT ("Bidirectional Encoder Representations from Transformers") architecture as its pre-trained language model. BERT-NER introduces the BERT installation to the NER mission making the context extraction be on the dot. BERT-NER fuses the BERT encoder and token classification layers together, optimized using cross-entropy loss. The experiment on the CoNLL-2003 dataset showed that BERT-NER could be an attention grabber as the scores were 93.12% precision, 94.46% recall, 93.79% F1 score, and 98.51% accuracy marking it as one of the field's top results. The qualitative evaluation on the other hand, shows brilliance in systemic solutions across multiple domains. This project not only is about named entity recognition but focuses on creating broader domain in order to enhance the flexibility of the process. Through benefiting from transfer learning and initialized language models, BERT-NER increases the capability of the BERT model, in this manner extending a list of its applications in NER.**

***Index Terms*—Named Entity Recognition (NER), Neural Networks, BERT, Tokenization, Classification**

I. Introduction

Named entity recognition (NER) is a key problem in the field of natural language processing (NLP) that involves searching out and categorizing named entities basically among people, places, organizations, and other specified information present within text corpora [1]. NER is a primary part of several NLP applications that implement more complex aspects like information retrieval, knowledge graph generation and question answering by removing meaningless and unstructured from unstructured text [2].

Fig. 1. NER Representation

Fig. 1 illustrates the workings of NER systems that underline and group named entities within a sentence. The person (PER) "Droupadi Murmu" is emphasized in red. "New Delhi" is colored blue to indicate that it represents a location (LOC). For organization (ORG), it is displayed in purple for "Bharat Heavy Electricals Limited". Other NE (O): Non-entity words

© 2024 The Authors. This is the author-accepted manuscript (post-print). Personal and research use of this material is permitted.
Published version available at IEEE Xplore: **https://doi.org/10.1109/ICCCNT61001.2024.10724703**

are shaded green. This system has the ability to identify and organize various types of proper nouns, common names and other linguistic constructs called named entities.

The focus of this research is the intricacies of neural network architectures with a particular emphasis on the development and implementation of a state-of-the-art NER model using BERT (Bidirectional Encoder Representations from Transformers). BERT, which provides deep bidirectional representations, represents a major leap forward in natural language processing and has completely transformed machine language understanding. BERT models have shown unmatched results across multiple NLP benchmarks through pre-training on a large corpus of text followed by task-specific fine-tuning.

The project "BERT-NER" falls under the category of these NLP systems, which are timely, flexible and efficient, thus can push forward the NLP community greatly. This research intention is to expand our initial research conducted on NER and develop widely applicable and flexible language comprehension systems. Language community remains dynamic and permanently in the process of effective communication.

II. Related Works

Yibo Wang et al. [3] provided a great framework that combines the multi-tasking ability to learn with machine reading comprehension, also known as MRC, which is used to improve Named Entity Recognition (NER) performance. The drawbacks of the Standard NER systems are addressed using this method, including their inability to manage nested or overlapping entities and their ability to take label relationships between various types of entities into account. The authors present a consolidated approach for both flat and nested NER problems by defining the NER as an MRC issue, where entity recognition is done by providing solutions relating to already established entity types.

The research paper by Ryan Cotterell and Kevin Duh [4] addressed the possible challenges of Named Entity Recognition (NER) in low-resource languages by using a cross-lingual approach. The paper also mentions the problems in obtaining high-performance NER systems in low-resource languages due to the shortage of elucidated data. The authors in this paper propose a transfer learning approach where character-level neural Conditional Random Fields also known as (CRFs) are trained across both high-resource and low-resource languages. This scheme uses language-based similarities between languages that are closely related and improves the performance of the NER system on languages that have no or less labelled data available.

Shuhe Wang and et al. [5] presents the method which is based on new way of named entity recognition (NER) using such great language models as Large Language Models (LLM). This research paper compares the skills and capabilities of conventional NER Tasks versus contemporary NER Machines versus Language Model (LLM) and puts light on the differences they notice. To expand the GPT-3 capability and reduce the accuracy degradation problem GPT-NER was designed. It is a new technique that converts the NER problem into a text-generation task, which addresses the current challenges and limitations of the existing LLMs.

The research paper by Weston et al. [6] addresses the problem of productively integrating new research findings along with knowledge in the materials science field. The study also offers a unique method for extracting and organizing data present in large numbers of materials science papers using the entity normalization algorithm along with the Named Entity Recognition (NER) algorithm. After this process of normalization, these entities exceed conventional text-based search techniques in terms of document retrieval. A web interface and the publicly available API make the structured data accessible, which results in allowing for simple data interaction and complex query capabilities.

A new unified framework for flat and nested NER tasks was proposed in the work of Xiaoya Li et al. [7]. The research treated NER as a machine reading comprehension task but not a sequence labeling problem. Explicitly, this work formalizes the extraction of the entities labeled as PERSON as extracting answer spans to the particular question "Which person is mentioned in the text? ". It allows generally inheriting the extraction of underlying entities, i.e., extracting two underlying entities into different divisions satisfying two autonomous queries. Further, since this solution explicitly includes informative prior knowledge in the query, it would lighten extraction for entities and in performance for both nested NER and flat NER.

Zhiheng Huang et al. [12] propose several models based on LSTMs for sequence tagging: simple LSTM networks, bidirectional LSTMs, LSTMs driven by a conditional random field layer, and, finally, the bidirectional variant of the latter architecture. This paper presents the first application of a BI-LSTM-CRF model to NLP benchmark sequence tagging datasets. The authors show that their BILSTM-CRF model, owing to a bidirectional LSTM component, will efficiently employ past and future input features. Because an extra layer of CRF is added, the model can use sentence-level tag information.

III. Contribution

This research aims to explore and contribute a Named Entity Recognition (NER) model that is versatile as well as resilient. Our goal includes gathering ample contextual information by using the abilities of pre-trained NLP models, specifically BERT. Achieving top-tier results on common NER standards is the main objective of this study. Based on this research, we hope and try to improve the NER methods to facilitate the creation of NLP systems that are way more flexible and adaptive.

IV. Methodology

A. BERT

In our work BERT-NER, the BERT model (Fig.2) is modified for Named Entity Recognition (NER), where a dataset including tagged words or tokens with associated entity categories is used to fine-tune the model.

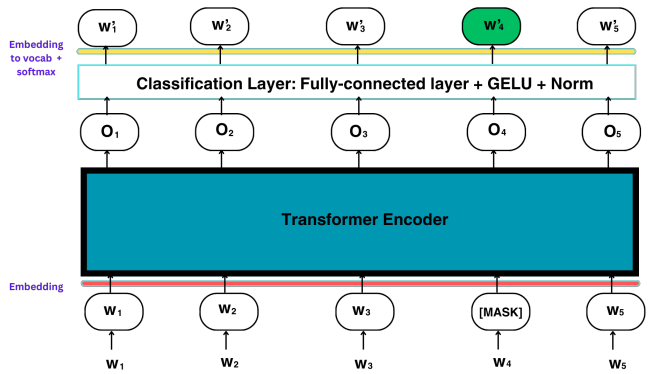

Fig. 2. BERT Architecture

a) Layer of Embedding: The embedding layer is the initial layer in the BERT model where each input token, W_1, W_2, W_3, $[MASK]$, W_5 for instance is converted into a vector which will be used as a representation of the word in high-dimensional space. To maintain sequence information that is critical for understanding the context of named entities within the same context window, an embedding layer combines token embeddings with positional embeddings.

For example(a), when the sentence "Elon drove to Chandigarh in his Tesla" is given to the model then words "Elon" "Chandigarh" and "Tesla" will be tokenized and their embeddings will capture not only semantic meaning but also their relative position.

The first stage of the embedding layer is conversion of each widget W_1 , W_2, W_3, MASK, W_5 into a dense vector form. Mathematically, this can be represented as:

$$E(W_i) = T(W_i) + P(i) \tag{1}$$

In equation 1, E(W_i) is the "vector representation of the i-th component", T(W_i) which is known as "Token embedding" and P(i) is the "vector corresponding to the position of 'i' in relation to the sequence". The subsequent word embedding tokens would be indexed from an embedding table $E \in \mathbb{R}^{VxD}$ where V stands for the "vocabulary size", and D is representative of the "size of the embedding".

b) Transformer Encoder: Certainly, the BERT model's heart is a Transformer encoder that processes input sequences. By employing self-attention mechanisms, it evaluates the significance of diverse tokens within an input sequence. It is crucial for tasks like NER which observe words as being meaningful based on surrounding words to understand the context of a whole sentence.

From the example(a), the Transformer encoder should understand from the context provided by the other words in the phrase that "Elon" is likely to be a person's name. At the same time "Chandigarh" is a place and "Tesla" is an organization.

The transformers' encoders are a stack of identical layers with a feed-forward neural network and, in turn, a self-attention mechanism. Here, it defines the mathematical model of the self-attention mechanism at layer:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2}$$

In equation 2, Q, K and V are query, key and value matrices respectively that are derived from input embeddings respectively and d_k represents a scaling factor which usually stands for key vector dimensions. This attention mechanism allows the model to weigh the influence of different tokens within the input sequence, considering the full context of the sentence.

c) Choosing Classes: The classes chosen by the model for each token are determined in the classification layer. It is made up of a fully- connected layer, GELU activation function and normalization unit. This part receives Transformer encoder's result and then gives every token a label like B-PER refers to "Beginning of Person Name", I-PER is "Internally within a Person Name", B- LOC is "Beginning of Location", I-LOC is "Inside Location", B- ORG is "Beginning of Organization Name" , I-ORG is "Inside an Organization Name", or O refers to "Outside any Named Entity".

For example, in this case, classification "Elon" could be outputted as being "B-PER", "Chandigarh" as "B-LOC" and "Tesla" as "B-ORG" while other labels would read O.

The classification layer always predicts entity type for each individual tokens. It has a fully connected layer followed by Gaussian Error Linear Unit (GELU) activation function and also with Normalization Layer. In final predictions, it takes the output from the Transformer encoder through this layer. Classification for each token can be represented as:

$$O_i = \text{Norm}(\text{GELU}(W_c \cdot H_i + b_c)) \tag{3}$$

In equation 3, the output O_i being the resultant value of the i-th token, W_c is the weighted matrix of the fully-connected layer, b_c is the bias term, H_i is the hidden state corresponding to the i-th token from the encoder layer of Transformer, and Norm is the layer normalization which are known for creating a balanced distribution of activations across the channels for the network.

d) Output Tokens: Output tokens O_1, O_2, O_3, O_4, O_5 show the final model predictions for each input token. In the context

of NER, these outputs correspond to the classified entities, being each token being labeled with an entity type e.g. B-PER, I-PER, O.

e) Special Tokens: For the MLM task (Masked Language Model) in the pre- training phase, there is a $[MASK]$ token structure provided on the input sequence in which the MLM is supposed to predict the identity of the masked word given the context in which it is placed. This is an important feature that enriches the models and ensures the best contextualization which leads to better results in NER.

B. Architecture

Labeling of Named Entities (NER) is one of the key NLP tasks where names are detected and tagged to predefined classes of the names such as first names, geographical names, names of institutions, times, amounts of money expressed as decimals or nouns or percentages. The code comes with a flavour of the modified version of the BERT model used especially for NER function.

Fig. 3 shows BertTokenizerFast is utilized for tokenization and term segmentation of the input text into words including the symbols for unknown words located in the beginning of the NER model.

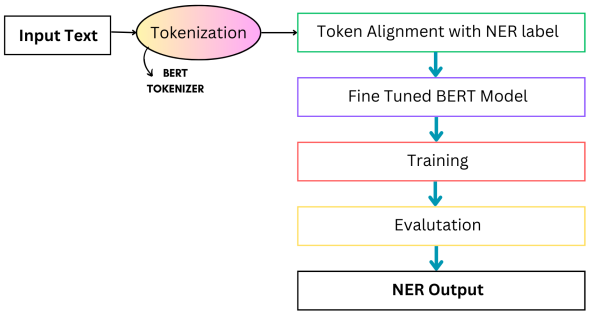

Fig. 3. BERT-NER Architecture

The NER model retails its workflow with preprocessing and tokenization by BertTokenizerFast that converts the input text into sub word tokens for unknown words. Specially designed CLS token is used to mark sequence division, and SEP token is specific for sequence completion. Labels are aligned with tokens and NER labels different from ones of special tokens are excluded during training time to prevent learning of irrelevant information.

The architecture of the model relies on Auto Model for Token Classification, employing BERT pre-trained model ("bert-base- uncased" in particular). Using Transformer encoders with self- attention mechanisms, it is able to get the general context around each token, which is followed by a sequence of hidden states that correspond to the input tokens.

C. Optimization

The cross-entropy loss, often known as NER, is a widely used loss-function for general classification tasks of tokenized datasets. This is the model that will be demonstrated by using the loss-function in the code. For a classification problem, the cross-entropy loss is defined as follows: The following is true in the event of cross-entropy loss in a classification task:

$$L = -\sum_{i=1}^{N} y_i \log(p_i) \tag{4}$$

In equation 4, N is known to be the "number of classes", y_i is the "binary indicator (0 or 1) if class label i is the correct classification for the observation", and p_i is the "predicted probability of the observation that belongs to class i".

For the NER task, where each token can belong to one of several classes (e.g., person, location, organization, etc.), the loss is computed for each token independently, and the average loss across all tokens in a batch is used to update the model parameters.

In the proposed model the cross-entropy loss is used in the token classification layer to ensure that the model differentiates between different entities.

V. Experiments

A. Dataset

BERT-NER is trained using the Conll-2003 dataset. CoNLL-2003 is one of the sampled datasets that are commonly used to evaluate the performance of NER tasks. It has labelled sentences that are marked with verified named entities. It was released as part of the CoNLL-2003 shared task and includes data for two languages: British and German. The dataset consists of a "training set", a "development set", a "testing set", and, in large size, unannotated data for each language. This dataset is vital for the creation and standardization of NER systems. They have been used and proven efficient to create a nicely labelled dataset of named entities. The statistics of the data used in the model training are shown in Table I.

TABLE I
Statistics of the datasets used

Dataset Name	Train	Validation	Test
CoNLL-2003	14041	3250	3453

B. Evaluation

The evaluation metrics for the BERT-NER model, which utilizes the BERT as its backbone, are given below(Table II):

Fig. 4 shows the validation accuracy of the model, which sharply increases at the first epoch and then slightly decreases

TABLE II
PERFORMANCE METRICS FOR THE BERT-NER MODEL

Model	Precision	Recall	F1	Accuracy
BERT-NER	93.12	94.46	93.79	98.51

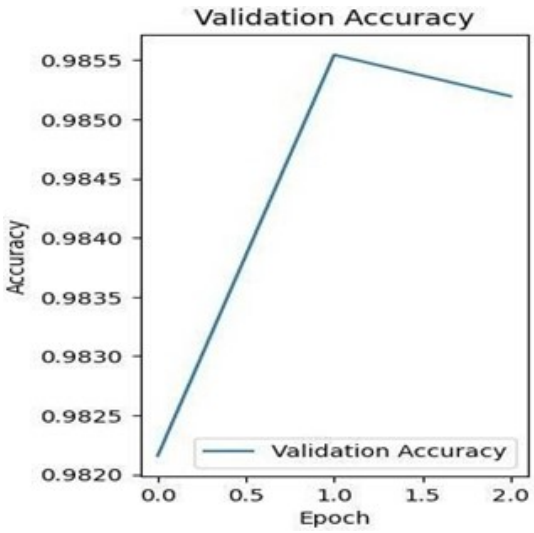

Fig. 4. Validation Accuracy

by the second epoch, indicating a peak in performance followed by a minor drop.

Fig. 5: Model loss during training; this is shown with the blue line representing train loss and an orange line indicating validation loss. The training loss starts high, about 0.225, which shows a poor fit of the model at its initial steps on training data. It finally hits about 0.025 near the end of the epochs, thus showing it has learned. The validation loss is much lower, around 0.075, where it decreases at a more gradual rate and hence stabilize around 0.05. This means the model generalizes relatively well to the validation data.

TABLE III
PRECISION OF PROPOSED MODEL WITH TOP-PERFORMANCE SYSTEMS

Model	Precision
BERT-MRC (2020) [7]	92.33
Ratinov and Roth (2009) [8]	91.20
Luo et al. (2015) [9]	91.50
Chiu and Nichols (2016) [10]	91.39
BERT-NER	**93.12**

TABLE IV
RECALL OF PROPOSED MODEL WITH TOP-PERFORMANCE SYSTEMS

Model	Recall
Ratinov and Roth (2009) [8]	90.50
Luo et al. (2015) [9]	91.40
Chiu and Nichols (2016) [10]	91.85
BERT-NER	**94.46**

From the tables (III, IV, and V), it was observed that BERT-NER performed better as compared to some top performance models in precision, recall, and f1 score.

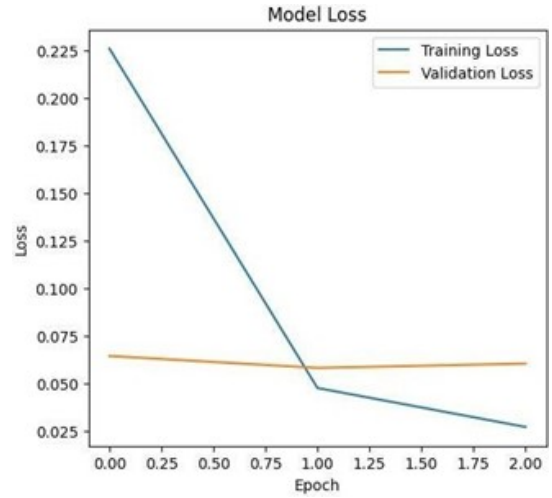

Fig. 5. Model Loss

TABLE V
F1 SCORE OF PROPOSED MODEL ON CONLL-2003 DATASET WITH PREVIOUS TOP-PERFORMANCE SYSTEMS

Model	F1
BERT-MRC (2020) [7]	93.04
Collobert et al. (2011) [11]	89.59
Huang et al. (2015) [12]	90.10
Chiu and Nichols (2016) [10]	91.62
BERT-NER	**93.79**

C. Robustness to Line Segmentation Errors

Among NER's problems, a misidentified line break in the input text affects performance heavily for models that optimize line by line. Although, two factors make BERT-NER less sensitive to such errors.

First, BERT employs WordPiece tokenization that means the word is split into pieces. Thus, if the line breaks exist in the middle of the word, English subwords can be matched correctly. For example (Fig. 6)

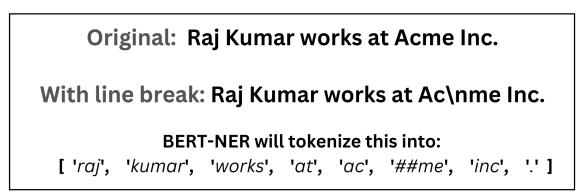

Fig. 6. BERT-NER Tokenization

The '##me' subword will still be correctly tagged and mapped back to 'Acme'.

Secondly, BERT-NER processes the input as a whole sequence and not individual lines as other techniques. By this way, the self attention makes it possible for the network to capture the complete context of the sentence.

VI. Results

It shows in Fig. 7 the result of applying the BERT-NER model to “Elon Musk is the founder of Tesla”. The assignment was identifying and classifying entities in the text into predefined categories, like a person or organization.

```
nlp = pipeline("ner", model=model_fine_tuned, tokenizer=tokenizer)

example = "Elon Musk is the founder of Tesla"

ner_results = nlp(example)

print(ner_results)

[{'entity': 'B-PER', 'score': 0.9976659, 'index': 1, 'word': 'el', 'start': 0, 'end': 2}, {'entity': 'B-PER',

'score': 0.998486, 'index': 2, 'word': '##on', 'start': 2, 'end': 4}, {'entity': 'I-PER', 'score': 0.99776745,

'word': 'mu', 'start': 5, 'end': 7}, {'entity': 'I-PER', 'score': 0.99721885, 'index': 4, 'word': '##sk', 'start': 7, 'end': 9}, {'

0.9868939, 'index': 9, 'word': 'tesla', 'start': 28, 'end': 33}]
```

Fig. 7. Model Output

This output is a list of dictionaries, each for the token in the input sentence classified as belonging to some named entity. Keys in these dictionaries contain information as follows: The entity predicted for the token, prefixed by ‘B-’ “at the beginning of an entity, ‘I-’ “inside an entity”, and ‘O-’ “outside an entity”. In this case, there is no O- entities since the output consists of only identified entities; score: The model’s confidence score regarding its prediction for this token; index: Position of token in sequence of input tokens; word: The actual token from the input sentence. Note that the model uses subword tokenization, which can split unknown or rare words into smaller pieces indicated with . Start: It reflects the starting character index of the token in the input sentence. End: It shows the end character index of the token in the input sentence.

The model identifies two entities in this sentence: “Elon Musk” as a person and “Tesla” as an organization. Here, the model is applying a BIO tagging scheme. “B-” means that this is the entity’s beginning, and “I-” means this token is inside the same entity. Because of subword tokenization, tokens like “Elon” and “Musk” were split into subtokens such as “el,” “on,” “mu”, and “sk”. The model correctly tagged “el” and “on” as the beginning of a person entity B-PER, while “mu” and “sk” are inside a person entity I-PER; “tesla” was also correctly identified as belonging to the beginning of an organization’s entity B-ORG.

The confidence scores are all high (above 0.98), indicating that the BERT-NER model is quite certain about its predictions. The “start” and end “indices” show where each token appears in the original sentence, which is useful for highlighting or extracting the entities from the text.

VII. Discussion and Future Work

A. Robustness to Noise

The proposed idea is also not sensitive to line segmentations that may have been done erroneously. As with other noise, BERT-NER accurately produces results when others include misspellings or non-standard words. This is so because, before the pre-training of the BERT model, it was trained from scratch on a large raw text data corpus, which by default contains much noise. The WordPiece tokenization also helps because it can split the unknown words into reasonable subwords if the separator is used in this case.

B. Token Limitations and Computational Complexity

Nonetheless, BERT-NER has a limitation on the maximum number of tokens for input, limited to 512 tokens via WordPiece subwords. This could be an issue for long documents, such as books, especially if they are rather large but seem sufficient for most sentences or short paragraphs.

Regarding efficiency, it can hardly be questioned that BERT-NER is more time-consuming and computationally intensive compared to the traditional NER frameworks, primarily because of BERT model usage. The majority of GPU resources are employed in fine-tuning and inference.

C. Future Work

Future work could probably try to devise how to extend the BERT architecture so that it can handle longer sequences more efficiently than the current architecture by using sparse attention mechanisms or hierarchical mechanisms. Other methods could also be used to create versions of the desired model that are lighter but with equally good performance; some of them are quantization and pruning.

VIII. Conclusion

In this study, we introduced BERT-NER, a method for Named Entity Recognition that makes use of the BERT language model that has already been trained. BERT-NER efficiently gathers contextual information to identify and categorize named things by optimizing BERT on NER tasks and adding a token classification layer. The BERT-NER works well in the experiments on the CoNLL-2003 benchmark dataset with a precision rate of 93.12%, a recall rate of 94.46%, and an accuracy rate of 98.51%, which efficiently outperforms other neural networks, reaching state-of-the-art performance. Results from this study are relevant in two broad aspects: general to NER and particularly to NLP. The methodology proposed herein opens new opportunities for designing adaptable yet durable and appropriate NER systems to deal with the variety and dynamics of real-world situations. The contribution of this work is to enable a new state-of-the-art score of 93.79% F1 on the CoNLL-2003 dataset, beating some previous approaches.

REFERENCES

[1] Jun-Yu Ma, Beiduo Chen, Jia-Chen Gu, Zhen-Hua Ling, Wu Guo Quan Liu, Zhigang Chen, Cong Liu. "WIDER CLOSER: Mixture of Short-channel Distillers for Zero-shot Cross-lingual Named Entity Recognition" (EMNLP-2022).

[2] Zihan Liu, Yan Xu, Tiezheng Yu, Wenliang Dai, Ziwei Ji, Samuel Cahyawijaya, Andrea Madotto, Pascale Fung. "CrossNER: Evaluating Cross-Domain Named Entity Recognition" (AAAI-2021).

[3] Yibo Wang, Wenting Zhao, Yao Wan, Zhongfen Deng. "Named Entity Recognition via Machine Reading Comprehension: A Multi-Task Learning Approach" (IJCNLP- AACL-2023).

[4] Ryan Cotterell, Kevin Duh. "Low-Resource Named Entity Recognition with Cross-Lingual, Character-Level Neural Conditional Random Fields" (IJCNLP 2017).

[5] Shuhe Wang, Xiaofei Sun, Xiaoya Li, Rongbin Ouyang, Fei Wu, Tianwei Zhang, Jiwei Li, Guoyin Wang. "GPT-NER: Named Entity Recognition via Large Language Models" (arXiv:2304.10428v4-2023).

[6] Leigh Weston, Vahe Tshitoyan, John Dagdelen, Olga Kononova, Kristin Persson, Gerbrand Ceder, Anubhav Jain. "Named Entity Recognition and Normalization Applied to Large-Scale Information Extraction from the Materials Science Literature" (JCIM-2019).

[7] Xiaoya Li, Jingrong Feng, Yuxian Meng, Qinghong Han, Fei Wu, and Jiwei Li. "A unified MRC framework for named entity recognition" (TACL-2020).

[8] Lev Ratinov and Dan Roth. "Design Challenges and Misconceptions in Named Entity Recognition" (TACL-2009).

[9] Gang Luo, Xiaojiang Huang, Chin-Yew Lin, and Zaiqing Nie. "Joint Entity Recognition and Disambiguation" (TACL-2015).

[10] Jason PC Chiu and Eric Nichols. "Named Entity Recognition with Bidirectional LSTM-CNNs" (TACL-2016).

[11] Ronan Collobert, Jason Weston, Leon Bottou, Michael Karlen, Koray Kavukcuoglu, and Pavel Kuksa. "Natural language processing (almost) from scratch" (JMLR-2011).

[12] Zhiheng Huang, Wei Xu, and Kai Yu. "Bidirectional LSTM-CRF models for sequence tagging" (arXiv:1508.01991-2015).